Chapter 10 Modelos de regressão dinâmica

▶ Carregar bibliotecas e configurações comuns

▶ Carregar bibliotecas adicionais

Os modelos de séries temporais dos dois capítulos anteriores permitem a inclusão de informações de observações passadas de uma série, mas não a inclusão de outras informações que também podem ser relevantes. Por exemplo, os efeitos de feriados, atividades da concorrência, mudanças na legislação, a situação econômica em geral ou outras variáveis externas podem explicar parte da variação histórica e levar a previsões mais precisas. Por outro lado, os modelos de regressão do Capítulo 7 permitem a inclusão de muitas informações relevantes de variáveis preditoras, mas não permitem a representação da dinâmica sutil das séries temporais que pode ser tratada com modelos ARIMA. Neste capítulo, consideramos como estender os modelos ARIMA para permitir a inclusão de outras informações.

No Capítulo 7 , consideramos modelos de regressão da forma

$$y_t = \beta_0 + \beta_1 x_{1\,,\,t} + \cdots + \beta_k x_{k\,,\,t} + \varepsilon_t,$$

ondey_té uma função linear de$k$variáveis preditoras ($x_{1\,,\,t}, \ldots, x_{k\,,\,t}$), e$\varepsilon_t$Geralmente, assume-se que o termo de erro resultante é não correlacionado (ou seja, ruído branco). Consideramos testes como o teste de Ljung-Box para avaliar se os resíduos resultantes eram significativamente correlacionados.

Neste capítulo, permitiremos que os erros de uma regressão contenham autocorrelação. Para enfatizar essa mudança de perspectiva, iremos substituir$\varepsilon_t$com$\eta_t$na equação. A série de errosη_t assume-se que segue um modelo ARIMA. Por exemplo, se$\eta_t$segue um modelo ARIMA(1,1,1), podemos escrever

$$\begin{aligned} y_t &= \beta_0 + \beta_1 x_{1\,,\,t} + \cdots + \beta_k x_{k\,,\,t} + \eta_t, \\ (\,1 - \phi_1 B\,)\,(\,1 - B\,)\,\eta_t &= (\,1 + \theta_1 B\,)\,\varepsilon_t, \end{aligned}$$

ondeε_tÉ uma série de ruído branco.

Observe que o modelo possui dois termos de erro aqui — o erro do modelo de regressão, que denotamos porη_te o erro do modelo ARIMA, que denotamos porε_tApenas os erros do modelo ARIMA são considerados ruído branco.

10.1 Estimativa

Ao estimarmos os parâmetros do modelo, precisamos minimizar a soma dos quadrados.ε_t valores. Se minimizarmos a soma dos quadradosη_tEm vez disso, os valores (que é o que aconteceria se estimássemos o modelo de regressão ignorando as autocorrelações nos erros) causam diversos problemas.

1. Os coeficientes estimados$\hat{\beta}_0, \ldots, \hat{\beta}_k$já não são as melhores estimativas, pois algumas informações foram ignoradas no cálculo;
2. Quaisquer testes estatísticos associados ao modelo (por exemplo, testes t nos coeficientes) estarão incorretos.
3. Os valores de AICc dos modelos ajustados já não são um bom indicador de qual é o melhor modelo para previsão.
4. Na maioria dos casos, opOs valores associados aos coeficientes serão muito pequenos, e, portanto, algumas variáveis preditoras parecerão importantes quando não o são. Isso é conhecido como "regressão espúria".

Minimizar a soma dos quadradosε_tO método dos valores evita esses problemas. Alternativamente, pode-se usar a estimação de máxima verossimilhança, que fornecerá estimativas semelhantes dos coeficientes.

Uma consideração importante ao estimar uma regressão com erros ARMA é que todas as variáveis do modelo devem ser estacionárias. Portanto, primeiro precisamos verificar sey_te todos os preditores$(x_{1,t}, \ldots, x_{k,t})$parecem ser estacionárias. Se estimarmos o modelo quando alguma delas for não estacionária, os coeficientes estimados não serão estimativas consistentes (e, portanto, podem não ser significativos). Uma exceção a isso ocorre quando as variáveis não estacionárias são cointegradas. Se existir uma combinação linear das variáveis não estacionáriasy_te os preditores que são estacionários, então os coeficientes estimados serão consistentes. [1]

Portanto, primeiro diferenciamos as variáveis não estacionárias no modelo. Muitas vezes é desejável manter a forma da relação entrey_te os preditores, e, consequentemente, é comum diferenciar todas as variáveis se alguma delas precisar de diferenciação. O modelo resultante é então chamado de “modelo em diferenças”, em contraste com um “modelo em níveis”, que é o que se obtém quando os dados originais são usados sem diferenciação.

If all of the variables in the model are stationary, then we only need to consider an ARMA process for the errors. It is easy to see that a regression model with ARIMA errors is equivalent to a regression model in differences with ARMA errors. For example, if the above regression

model with ARIMA(1,1,1) errors is differenced we obtain the model

$$y'_t = \beta_1 x'_{1,t} + \cdots + \beta_k x'_{k,t} + \eta'_t,$$
$$(1 - \phi_1 B)\eta'_t = (1 + \theta_1 B)\varepsilon_t,$$

where $y'_t = y_t - y_{t-1}$, $x'_{t,i} = x_{t,i} - x_{t-1,i}$ and $\eta'_t = \eta_t - \eta_{t-1}$, which is a regression model in differences with ARMA errors.

10.2 Regression with ARIMA errors using `StatsForecast`

The class `ARIMA()` will fit a regression model with ARIMA errors if exogenous regressors are included in the dataframe. As introduced in Section 9.5, `order` specifies the order of the ARIMA error model following the convention `(p, d, q)`. If differencing is specified, then the differencing is applied to all variables in the regression model before the model is estimated. For example, if `train_df` contains one additional exogenous variable, the following code

```
sf = StatsForecast(models=[ARIMA(order=(1, 1, 0))])
sf.fit(df=train_df)
```

will fit the model $y'_t = \beta_1 x'_t + \eta'_t$, where $\eta'_t = \phi_1 \eta'_{t-1} + \varepsilon_t$ is an AR(1) error. This is equivalent to the model

$$y_t = \beta_0 + \beta_1 x_t + \eta_t,$$

where η_t is an ARIMA(1,1,0) error. Notice that the constant term disappears due to the differencing. To include a constant in the differenced model, we would specify `ARIMA(..., include_constant=True)`.

The `AutoARIMA()` can be used to select the best ARIMA model for the errors. Whether differencing is required is determined by applying a KPSS test to the residuals from the regression model estimated using ordinary least squares. If differencing is required, then all variables are differenced and the model re-estimated using maximum likelihood estimation. The final model will be expressed in terms of the original variables, even if it has been estimated using differenced variables.

The AICc is calculated for the final model, and this value can be used to determine the best predictors. That is, the procedure should be repeated for all subsets of predictors to be considered, and the model with the lowest AICc value selected.

Example: US Personal Consumption and Income

Figure 10.1 shows the quarterly changes in personal consumption expenditure and personal disposable income from 1970 to 2019 Q2. We would like to forecast changes in expenditure based

on changes in income. A change in income does not necessarily translate to an instant change in consumption (e.g., after the loss of a job, it may take a few months for expenses to be reduced to allow for the new circumstances). However, we will ignore this complexity in this example and try to measure the instantaneous effect of the average change of income on the average change of consumption expenditure.

```
us_change = (
    pd.read_csv('../data/US_change.csv', parse_dates=["ds"])
    .rename(columns={"y": "Consumption"})
    [["unique_id", "ds", "Consumption", "Income"]]
)

fig, axes = plt.subplots(2, sharex=True)
sns.lineplot(data=us_change, x="ds", y="Consumption", color="black",
    label="Consumption", ax=axes[0])
sns.lineplot(data=us_change, x="ds", y="Income", color="black",
    label="Income", ax=axes[1])
axes[0].set(ylabel="")
axes[1].set(ylabel="", xlabel="Quarter")
fig.supylabel("Quarterly % change")
fig.suptitle("US consumption and personal income")
plt.show()
```

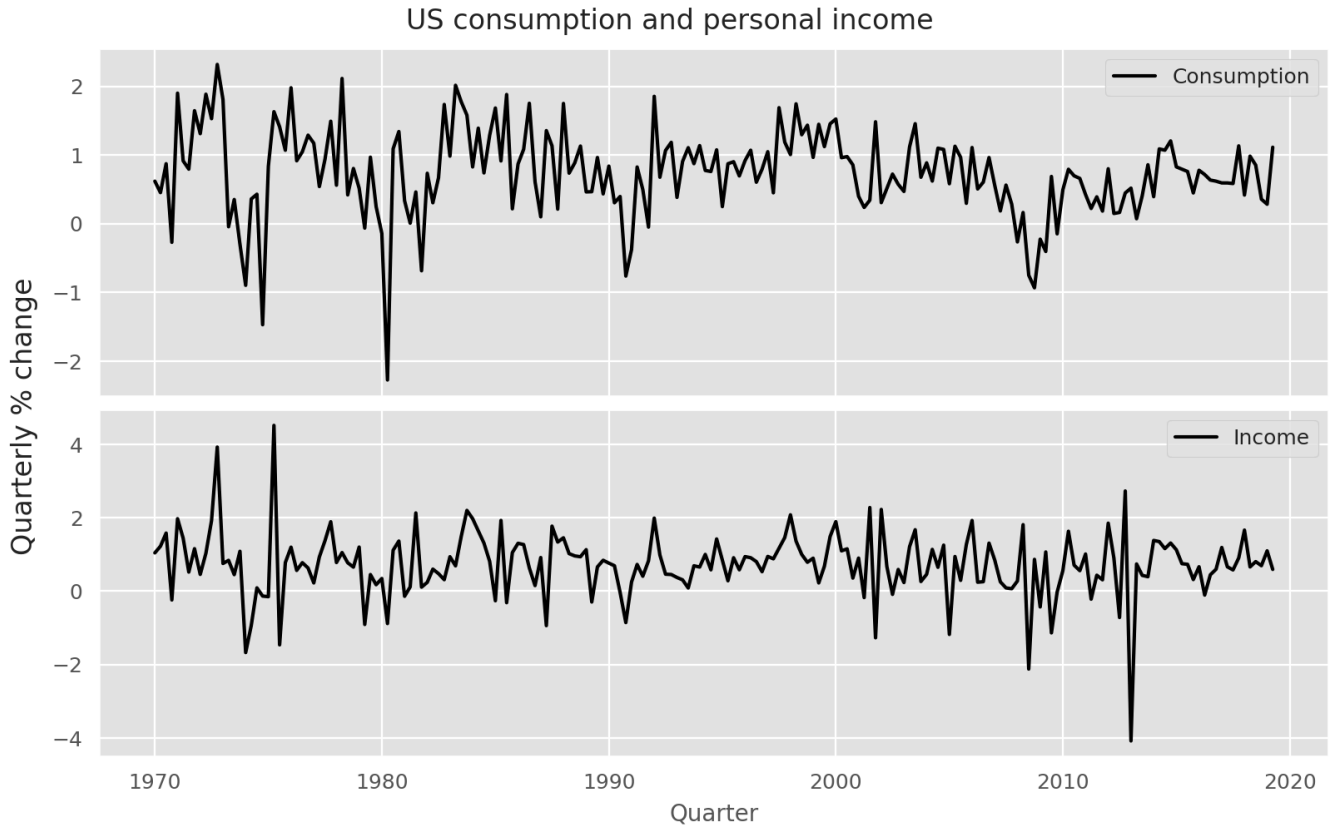

Figure 10.1: Percentage changes in quarterly personal consumption expenditure and personal disposable income for the USA, 1970 Q1 to 2019 Q2.

```
sf = StatsForecast(models=[AutoARIMA()], freq="QS")
sf = sf.fit(df=us_change, target_col="Consumption")
```

```
model = sf.fitted_[0, 0].model_
items = list(model["coef"].items())
arma = pd.Series(dict(items[:3]), name="ARMA Coefficients").round(3)
regr = pd.Series(dict(items[3:]), name="Regression Coefficients").round(3)
metrics = pd.Series({
    "sigma^2": model["sigma2"], "loglik": model["loglik"],
    "aic": model["aic"], "aicc": model["aicc"], "bic": model["bic"],
}, name="Metrics").round(2)
print(ARIMASummary(model), "\n")
print(arma.to_frame(), "\n")
print(regr.to_frame(), "\n")
print(metrics.to_frame())
```

```
Regression with ARIMA(1,0,2) errors

     ARMA Coefficients
ar1              0.707
ma1             -0.617
ma2              0.207

           Regression Coefficients
intercept                    0.595
ex_1                         0.198

         Metrics
sigma^2     0.31
loglik   -163.04
aic       338.07
aicc      338.51
bic       357.80
```

The data are clearly already stationary (as we are considering percentage changes rather than raw expenditure and income), so there is no need for any differencing. The fitted model is

$$
\begin{aligned}
y_t &= 0.595 + 0.198x_t + \eta_t, \\
\eta_t &= 0.707\eta_{t-1} + \varepsilon_t - 0.617\varepsilon_{t-1} + 0.207\varepsilon_{t-2}, \\
\varepsilon_t &\sim \mathrm{NID}(0, 0.311).
\end{aligned}
$$

We can recover estimates of both the η_t and ε_t series as shown in Figures 10.2 and Figure 10.3.

```
innov_resid = model["residuals"]
reg_resid = (us_change["Consumption"]
    - regr["ex_1"] * us_change["Income"]
    - regr["intercept"])
resid = pd.DataFrame({
    "ARIMA residuals": innov_resid,
    "Regression residuals": reg_resid,
    "ds": us_change["ds"],
})

fig, axes = plt.subplots(2, sharex=True)
sns.lineplot(data=resid, x="ds", y="Regression residuals", color="black",
```

```
    ax=axes[0])
sns.lineplot(data=resid, x="ds", y="ARIMA residuals", color="black",
    ax=axes[1])
axes[1].set(xlabel='Quarter')
plt.show()
```

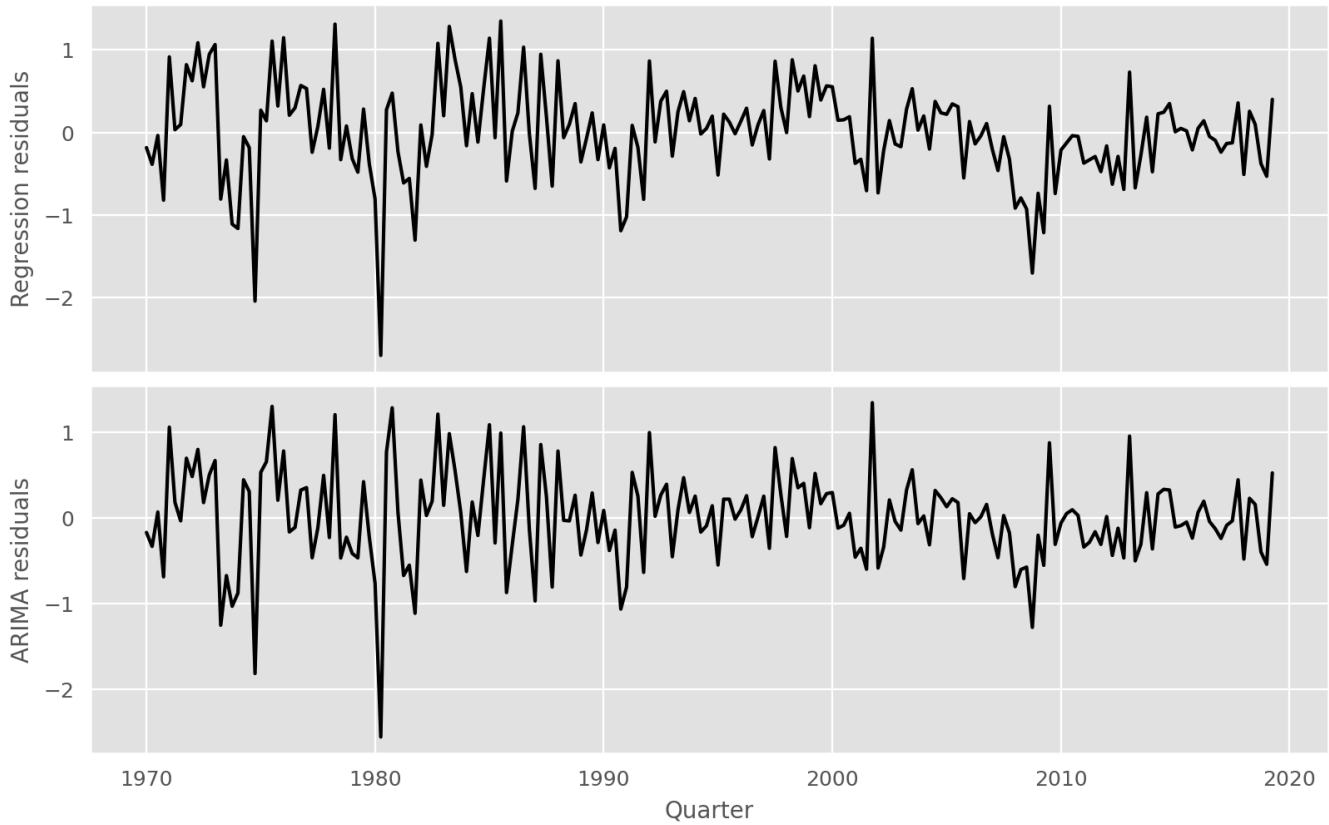

Figure 10.2: Regression residuals (η_t) and ARIMA residuals (ε_t) from the fitted model.

It is the ARIMA estimated errors (the innovation residuals) that should resemble a white noise series.

```
fig, axes = plt.subplot_mosaic([["resid", "resid"], ["acf", "hist"]])
axes["resid"].plot(resid["ds"], innov_resid, marker="o")
axes["resid"].set(xlabel="Quarter", ylabel="Innovation Residuals")
plot_acf(innov_resid, axes["acf"], zero=False, auto_ylims=True,
    bartlett_confint=False)
axes["acf"].xaxis.set_major_locator(MaxNLocator(integer=True))
axes["acf"].set(xlabel="lag [1Q]", ylabel="acf", title="")
axes["hist"].hist(innov_resid, bins=20)
axes["hist"].set(xlabel="residuals", ylabel="count")
plt.show()
```

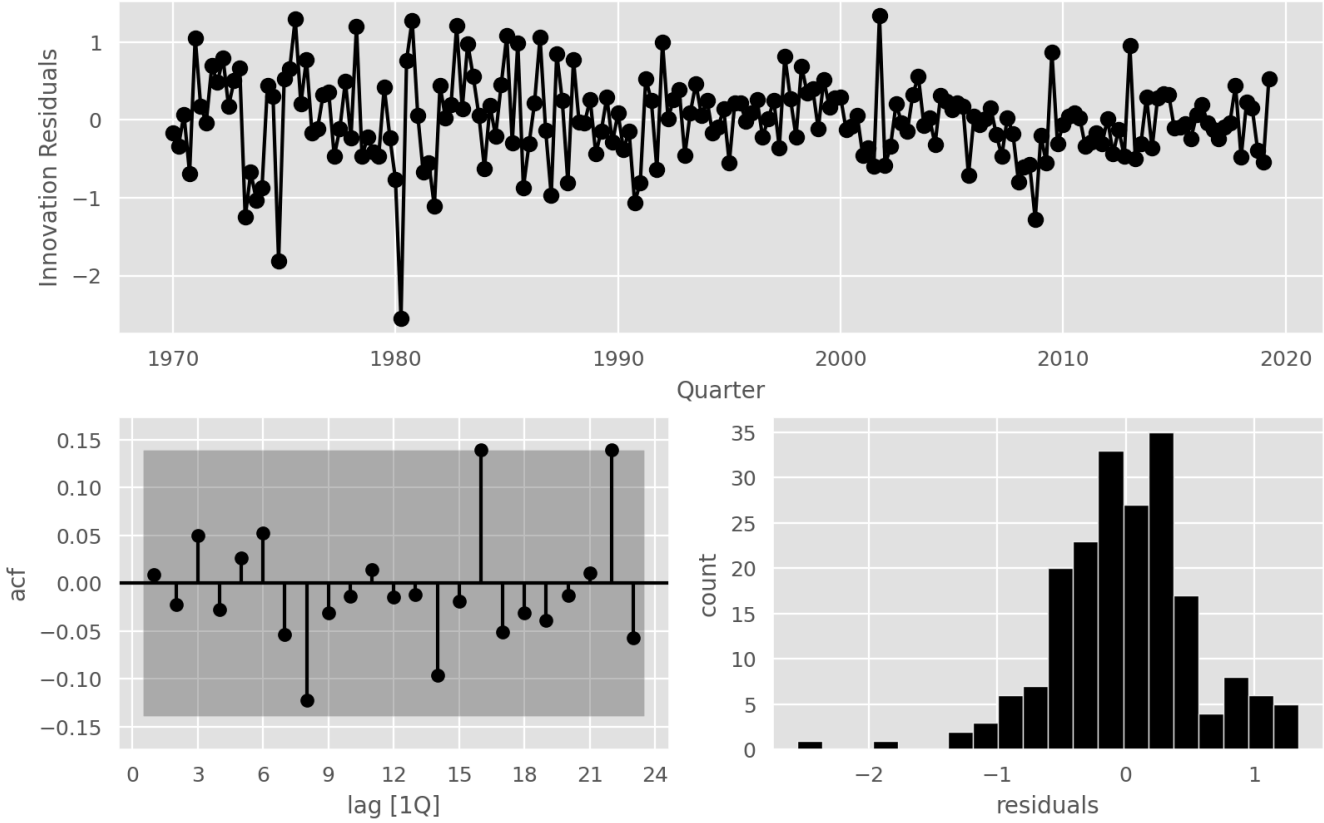

Figure 10.3: The innovation residuals (i.e., the estimated ARIMA errors) are not significantly different from white noise.

```
ljung_box = acorr_ljungbox(innov_resid, lags=[8], model_df=3)
ljung_box
```

	lb_stat	lb_pvalue
8	5.21	0.391

10.3 Forecasting

To forecast using a regression model with ARIMA errors, we need to forecast the regression part of the model and the ARIMA part of the model, and combine the results. As with ordinary regression models, in order to obtain forecasts we first need to forecast the predictors. When the predictors are known into the future (e.g., calendar-related variables such as time, day-of-week, etc.), this is straightforward. But when the predictors are themselves unknown, we must either model them separately, or use assumed future values for each predictor.

Example: US Personal Consumption and Income

We will calculate forecasts for the next eight quarters assuming that the future percentage changes in personal disposable income will be equal to the mean percentage change from the

last forty years.

```
sf = StatsForecast(models=[AutoARIMA()], freq="QS")
uids = us_change["unique_id"].unique()
last_times = us_change[["ds"]].max()
future_predictors = (
    make_future_dataframe(uids, last_times, freq="QS", h=8)
    .assign(Income=us_change["Income"].mean())
)
level = [80, 95]
fc = sf.forecast(df=us_change, h=8,
    X_df=future_predictors, target_col="Consumption",
    fitted=True, level=level)
plot_series(us_change, fc, target_col="Consumption", level=level,
    title="", xlabel="Quarter", ylabel="Percentage change",
    palette="black_and_blue", rm_legend=False)
```

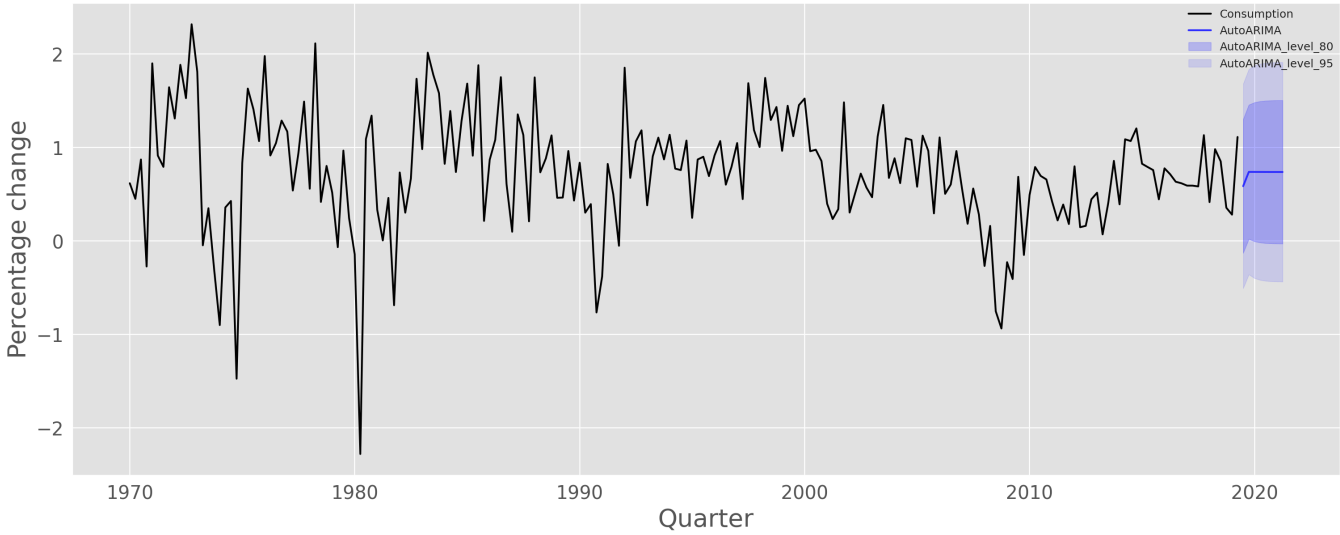

Figure 10.4: Forecasts obtained from regressing the percentage change in consumption expenditure on the percentage change in disposable income, with an ARIMA error model.

The prediction intervals for this model are narrower than if we had fitted an ARIMA model without covariates, because we are now able to explain some of the variation in the data using the income predictor.

It is important to realise that the prediction intervals from regression models (with or without ARIMA errors) do not take into account the uncertainty in the forecasts of the predictors. So they should be interpreted as being conditional on the assumed (or estimated) future values of the predictor variables.

Example: Forecasting electricity demand

Daily electricity demand can be modelled as a function of temperature. As can be observed on an electricity bill, more electricity is used on cold days due to heating and hot days due to air

conditioning. The higher demand on cold and hot days is reflected in the U-shape of Figure 10.5, where daily demand is plotted versus daily maximum temperature.

```
vic_elec = (
    pd.read_csv('../data/vic_elec.csv', parse_dates=['ds'])
    .loc[lambda x: x["ds"] >= "2014"]
    .pivot(index=["ds", "Holiday"], columns="unique_id", values="y")
    .reset_index(level="Holiday").rename_axis(None, axis="columns")
)
vic_elec_daily = (
    vic_elec
    .resample("D")
    .agg({"Demand": "sum", "Temperature": "max", "Holiday": "first"})
    .reset_index()
    .assign(
        unique_id="VIC",
        Demand=lambda x: x["Demand"] / 1e3,
        Weekend=lambda x: x["ds"].dt.dayofweek.isin([5, 6]),
        Day_Type=lambda x: np.select(
            [x["Holiday"], ~x["Holiday"] & x["Weekend"]],
            ["Holiday", "Weekend"],
            default="Weekday",
        )
    )
)

fig, ax = plt.subplots()
sns.scatterplot(data=vic_elec_daily,
    x="Temperature", y="Demand", hue="Day_Type")
ax.set(
    xlabel="Maximum daily temperature",
    ylabel="Electricity demand (GW)",
)
plt.show()
```

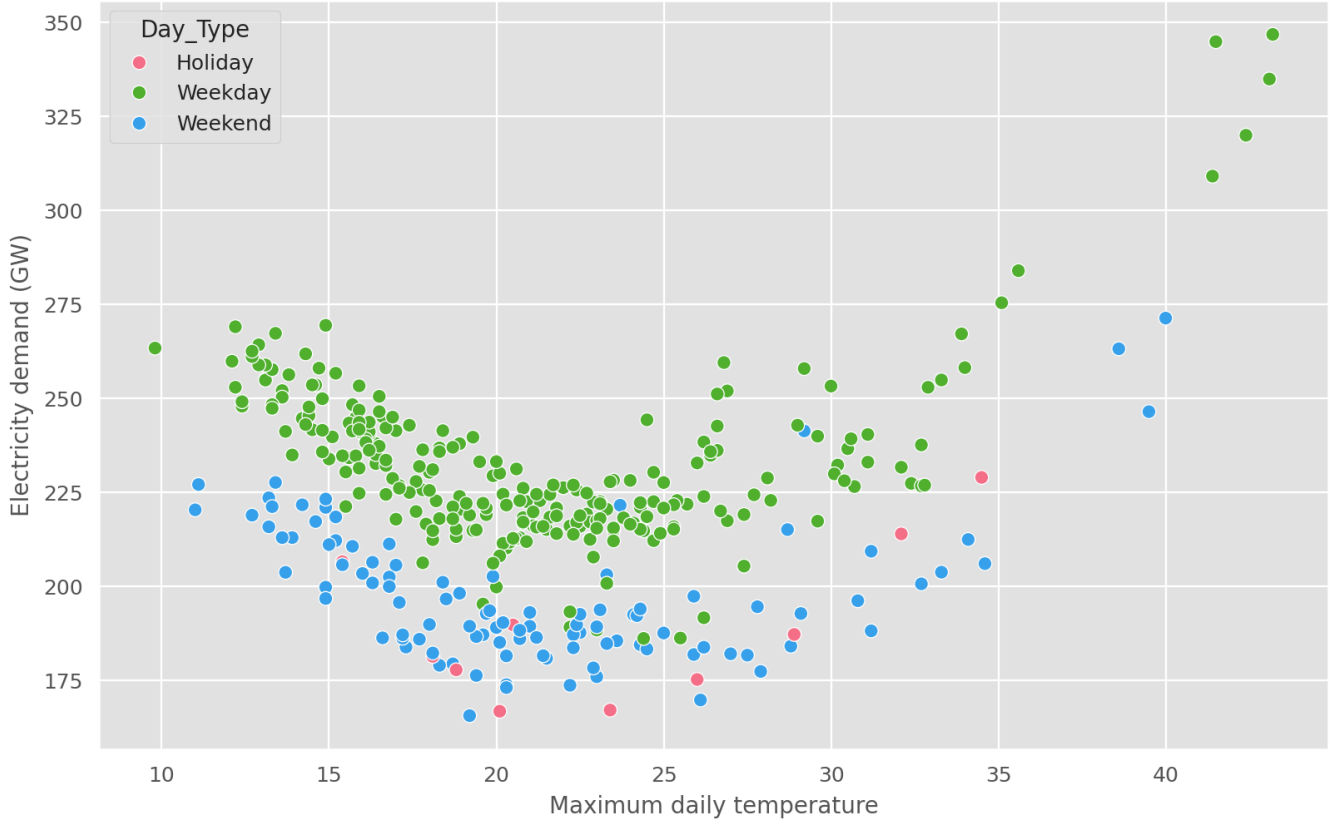

Figure 10.5: Daily electricity demand versus maximum daily temperature for the state of Victoria in Australia for 2014.

The data stored as `vic_elec_daily` includes total daily demand, daily maximum temperatures, and an indicator variable for if that day is a public holiday. Figure 10.6 shows the time series of both daily demand and daily maximum temperatures. The plots highlight the need for both a non-linear and a dynamic model.

```
fig, axes = plt.subplots(2, sharex=True)
sns.lineplot(data=vic_elec_daily, x='ds', y='Demand',
    color='black', ax=axes[0])
sns.lineplot(data=vic_elec_daily, x='ds', y='Temperature',
    color='black', ax=axes[1])
axes[1].set(xlabel="Date")
plt.show()
```

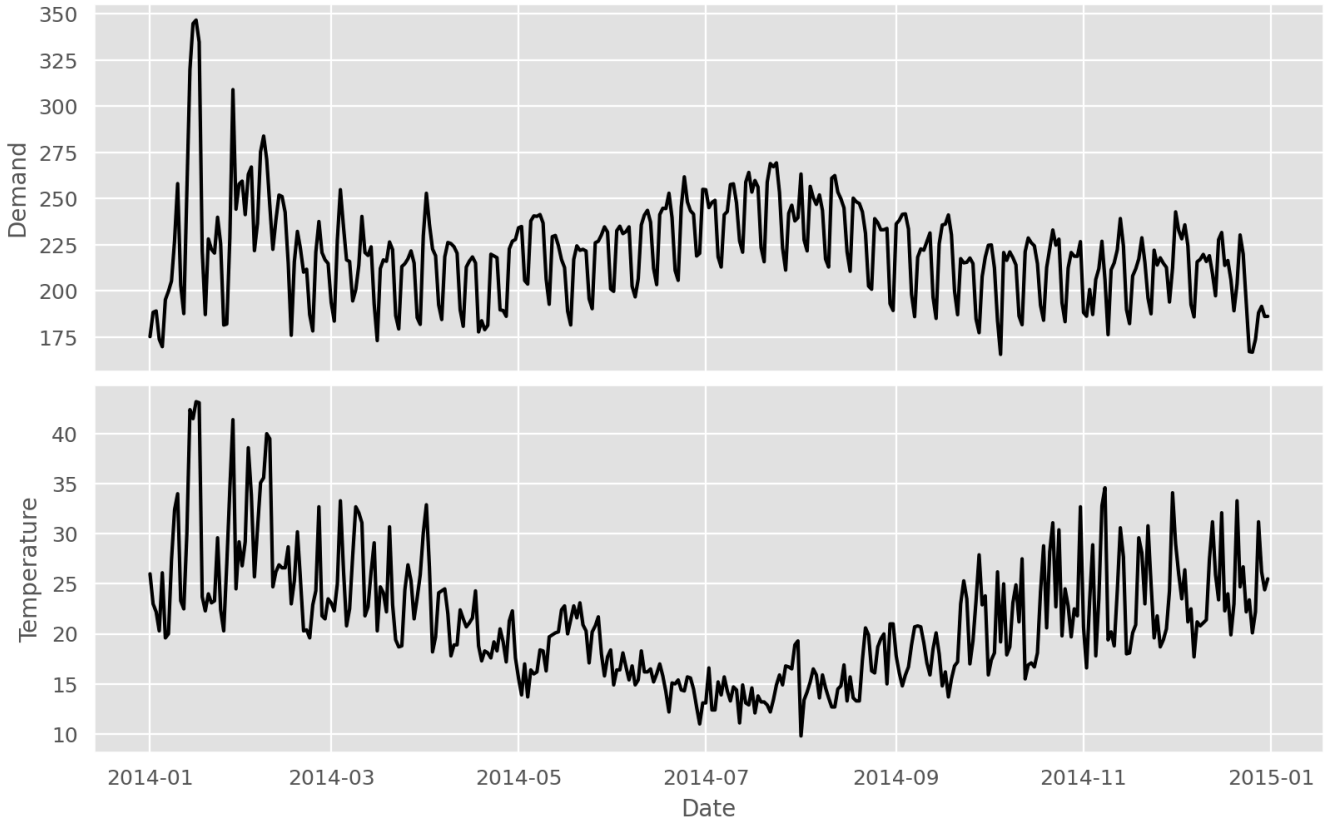

Figure 10.6: Daily electricity demand and maximum daily temperature for the state of Victoria in Australia for 2014.

In this example, we fit a quadratic regression model with ARMA errors using the `ARIMA()` function. The model also includes an indicator variable for if the day was a working day or not.

```
sf = StatsForecast(models=[AutoARIMA(season_length=7)], freq="D")
ex_columns = ["unique_id", "ds", "Demand", "Temperature"]
vic_elec_daily_ex = vic_elec_daily[ex_columns].assign(**{
    "Weekday": vic_elec_daily["Day_Type"] == "Weekday",
    "Temperature**2": vic_elec_daily["Temperature"]**2,
})
sf = sf.fit(df=vic_elec_daily_ex, target_col="Demand")
resids = sf.fitted_[0, 0].model_["residuals"]

fig, axes = plt.subplot_mosaic([["resid", "resid"], ["acf", "hist"]])
axes["resid"].plot(vic_elec_daily["ds"], resids, marker="o")
axes["resid"].set(xlabel="Date", ylabel="Innovation Residuals")
plot_acf(resids, axes["acf"], zero=False, auto_ylims=True,
    bartlett_confint=False)
axes["acf"].xaxis.set_major_locator(MaxNLocator(integer=True))
axes["acf"].set(xlabel="lag [1D]", ylabel="acf", title="")
axes["hist"].hist(resids, bins=20)
axes["hist"].set(xlabel="residuals", ylabel="count")
plt.show()
```

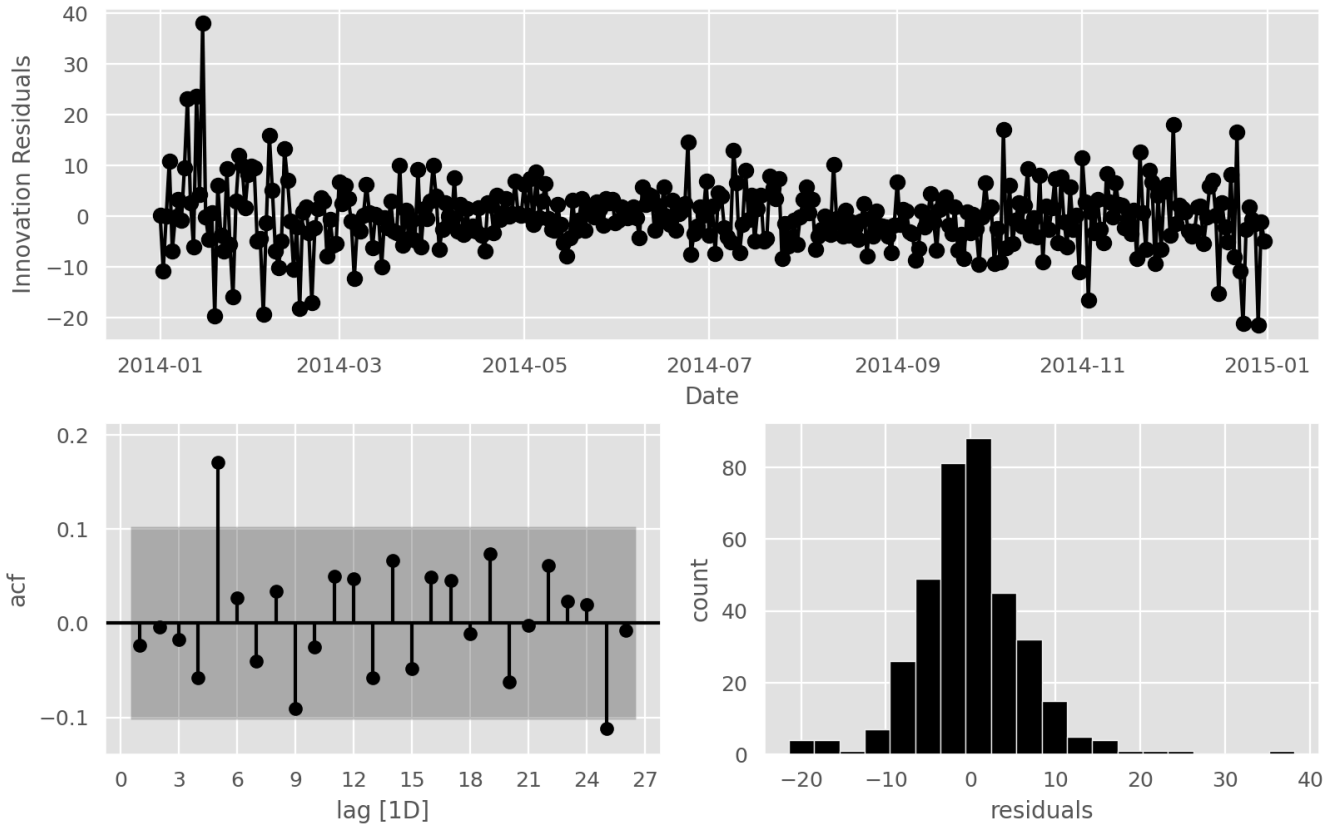

Figure 10.7: Residuals diagnostics for a dynamic regression model for daily electricity demand with workday and temperature effects.

```
model = sf.fitted_[0, 0].model_
items = list(model["coef"].items())
arma = pd.Series(dict(items[:8]), name="ARMA Coefficients").round(3)
regr = pd.Series(dict(items[8:]), name="Regression Coefficients").round(3)
metrics = pd.Series({
    "sigma^2": model["sigma2"], "loglik": model["loglik"],
    "aic": model["aic"], "aicc": model["aicc"], "bic": model["bic"],
}, name="Metrics").round(2)
print(ARIMASummary(model), "\n")
print(arma.to_frame(), "\n")
print(regr.to_frame(), "\n")
print(metrics.to_frame())
```

```
Regression with ARIMA(2,1,2)(2,0,2)[7] errors

      ARMA Coefficients
ar1              -0.087
ar2               0.716
ma1              -0.024
ma2              -0.914
sar1              0.309
sar2              0.566
sma1             -0.164
sma2             -0.386

      Regression Coefficients
ex_1                   -7.501
```

```
ex_2                    29.325
ex_3                     0.178

         Metrics
sigma^2    43.06
loglik  -1197.38
aic      2418.77
aicc     2419.66
bic      2465.53
```

The fitted model has an ARIMA(2,1,2)(2,0,2)[7] error, so there are 8 AR and MA coefficients.

```
ljung_box = acorr_ljungbox(resids, lags=[14], model_df=8)
ljung_box
```

	lb_stat	lb_pvalue
14	21.9	0.00128

There is clear heteroscedasticity in the residuals, with higher variance in January and February, and lower variance in May. The model also has some significant autocorrelation in the residuals, and the histogram of the residuals shows long tails. All of these issues with the residuals may affect the coverage of the prediction intervals, but the point forecasts should still be ok.

Using the estimated model we forecast 14 days ahead starting from Thursday 1 January 2015 (a non-work-day being a public holiday for New Year's Day). In this case, we could obtain weather forecasts from the weather bureau for the next 14 days. But for the sake of illustration, we will use scenario based forecasting (as introduced in Section 7.6) where we set the temperature for the next 14 days to a constant 26 degrees.

```
uids = vic_elec_daily_ex["unique_id"].unique()
last_times = vic_elec_daily_ex[["ds"]].max()
future_predictors = (
    make_future_dataframe(uids, last_times, freq="D", h=14)
    .assign(**{
        "Weekday": lambda x: ~x["ds"].dt.dayofweek.isin([5, 6]),
        "Temperature": 26,
        "Temperature**2": 26**2,
    })
)
level = [80, 95]
# Ensure column order agrees with usage above in sf.fit()
columns = vic_elec_daily_ex.columns.drop('Demand')
fc = sf.predict(h=14, X_df=future_predictors[columns], level=level)

plot_series(vic_elec_daily_ex, fc, target_col="Demand", level=level,
    xlabel="Date", ylabel="GW",
    title="Daily electricity demand: Victoria",
    palette="black_and_blue", rm_legend=False)
```

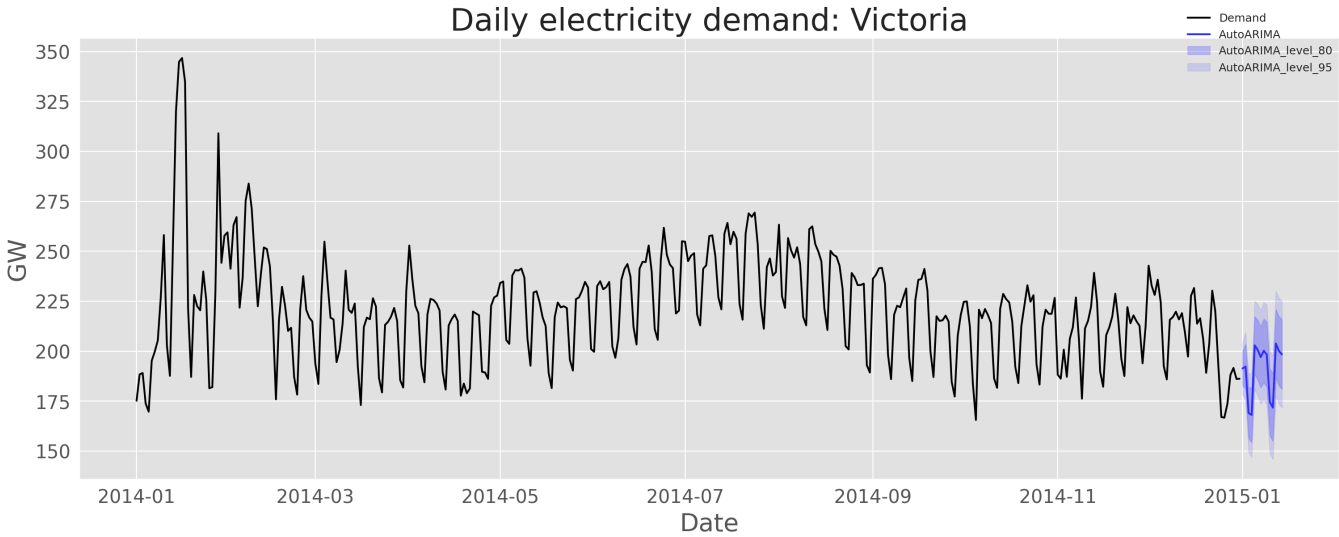

Figure 10.8: Forecasts from the dynamic regression model for daily electricity demand. All future temperatures have been set to 26 degrees, and the working day dummy variable has been set to known future values.

The point forecasts look reasonable for the first two weeks of 2015. The slow down in electricity demand at the end of 2014 (due to many people taking summer vacations) has caused the forecasts for the next two weeks to show similarly low demand values.

10.4 Stochastic and deterministic trends

There are two different ways of modelling a linear trend. A *deterministic trend* is obtained using the regression model

$$y_t = \beta_0 + \beta_1 t + \eta_t,$$

where η_t is an ARMA process. A *stochastic trend* is obtained using the model

$$y_t = \beta_0 + \beta_1 t + \eta_t,$$

where η_t is an ARIMA process with $d = 1$. In the latter case, we can difference both sides so that $y'_t = \beta_1 + \eta'_t$, where η'_t is an ARMA process. In other words,

$$y_t = y_{t-1} + \beta_1 + \eta'_t.$$

This is similar to a random walk with drift (introduced in Section 9.1), but here the error term is an ARMA process rather than simply white noise.

Although these models appear quite similar (they only differ in the number of differences that need to be applied to η_t), their forecasting characteristics are quite different.

Example: Air transport passengers Australia

```
aus_airpassengers = (
    pd.read_csv('../data/aus_airpassengers.csv', parse_dates=['Year'])
    [['Year', 'Passengers']]
    .rename(columns={'Year':'ds', 'Passengers':'y'})
    .assign(unique_id="aus_airpassengers")
)

plot_series(aus_airpassengers,
    xlabel="Year [1Y]", ylabel="Passengers (millions)",
    title="Total annual air passengers", palette="black")
```

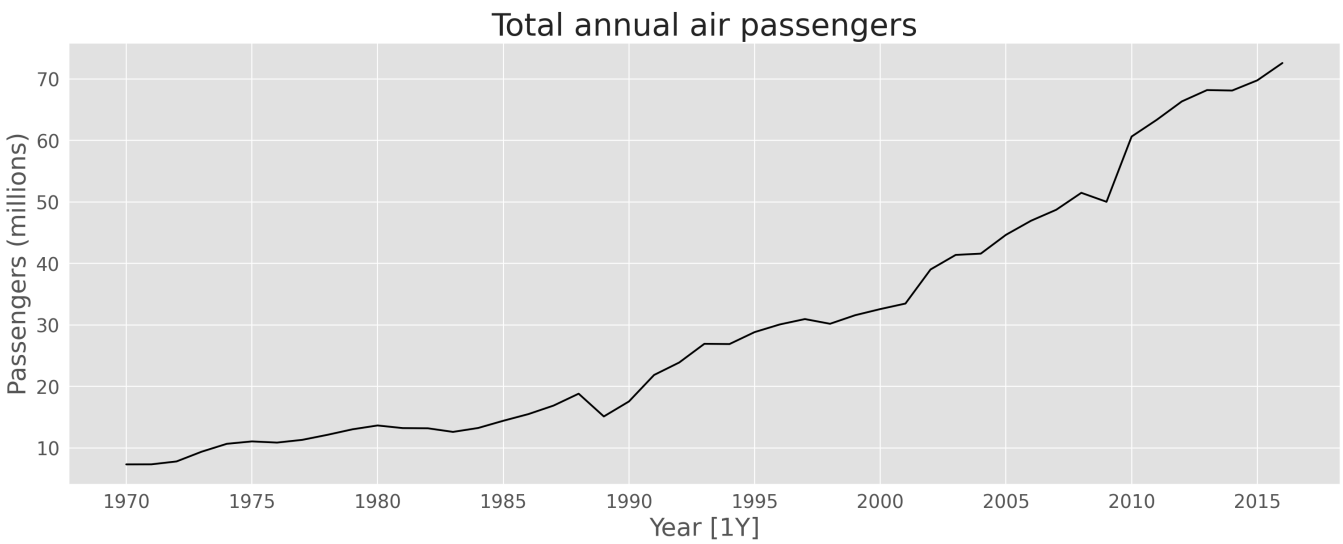

Figure 10.9: Total annual passengers (in millions) for Australian air carriers, 1970–2016.

Figure 10.9 shows the total number of passengers for Australian air carriers each year from 1970 to 2016. We will fit both a deterministic and a stochastic trend model to these data using the `trend` method from `utilsforecast`. The deterministic trend model is obtained as follows:

```
train_df, future_df = trend(
    aus_airpassengers, freq='YS', h=20)
train_df["intercept"] = 1
future_df["intercept"] = 1
model = AutoARIMA(d=0, alias="AutoARIMA_det")
sf = StatsForecast(models=[model], freq="YS")
sf = sf.fit(df=train_df)
fc_det = sf.predict(h=20, X_df=future_df, level=[80, 95])

model = sf.fitted_[0, 0].model_
coefs = {**model["coef"]}
coefs = pd.Series(coefs, name="Coefficients").round(3)
metrics = pd.Series({
    "sigma^2": model["sigma2"], "loglik": model["loglik"],
    "aic": model["aic"], "aicc": model["aicc"], "bic": model["bic"],
}, name="Metrics").round(2)

print(ARIMASummary(model), "\n")
```

```
print(coefs.to_frame(), "\n")
print(metrics.to_frame())
```

```
Regression with ARIMA(1,0,0) errors

      Coefficients
ar1          0.956
ex_1         1.415
ex_2         0.902

         Metrics
sigma^2     4.34
loglik   -100.88
aic       209.77
aicc      210.72
bic       217.17
```

This model can be written as

$$
\begin{aligned}
y_t &= 0.902 + 1.415t + \eta_t \\
\eta_t &= 0.956\eta_{t-1} + \varepsilon_t \\
\varepsilon_t &\sim \mathrm{NID}(0, 4.34).
\end{aligned}
$$

The estimated growth in visitor numbers is 1.42 million people per year.

Alternatively, the stochastic trend model can be estimated.

```
model = AutoARIMA(d=1, alias="AutoARIMA_stoch")
sf = StatsForecast(models=[model], freq="YS")
sf = sf.fit(df=aus_airpassengers)
fc_stoch = sf.predict(h=20, level=[80, 95])

model = sf.fitted_[0, 0].model_
coefs = {**model["coef"]}
coefs = pd.Series(coefs, name="Coefficients").round(3)
metrics = pd.Series({
    "sigma^2": model["sigma2"], "loglik": model["loglik"],
    "aic": model["aic"], "aicc": model["aicc"], "bic": model["bic"],
}, name="Metrics").round(2)

print(ARIMASummary(model), "\n")
print(coefs.to_frame(), "\n")
print(metrics.to_frame())
```

```
ARIMA(0,1,0) with drift

       Coefficients
drift         1.419

         Metrics
sigma^2     4.27
loglik    -98.16
aic       200.31
```

```
aicc       200.59
bic        203.97
```

This model can be written as $y_t - y_{t-1} = 1.419 + \varepsilon_t$, or equivalently

$$\begin{aligned} y_t &= y_0 + 1.419t + \eta_t \\ \eta_t &= \eta_{t-1} + \varepsilon_t \\ \varepsilon_t &\sim \mathrm{NID}(0, 4.27). \end{aligned}$$

In this case, the estimated growth in visitor numbers is also 1.42 million people per year. Although the growth estimates are similar, the prediction intervals are not, as Figure 10.10 shows. In particular, stochastic trends have much wider prediction intervals because the errors are non-stationary.

```
df = fc_det.merge(fc_stoch, on=['unique_id', 'ds'])
plot_series(aus_airpassengers, df, level=[95],
    xlabel="Year [1Y]", ylabel="Air passengers (millions)",
    title="Forecasts from trend models",
    palette="black_and_2color", rm_legend=False)
```

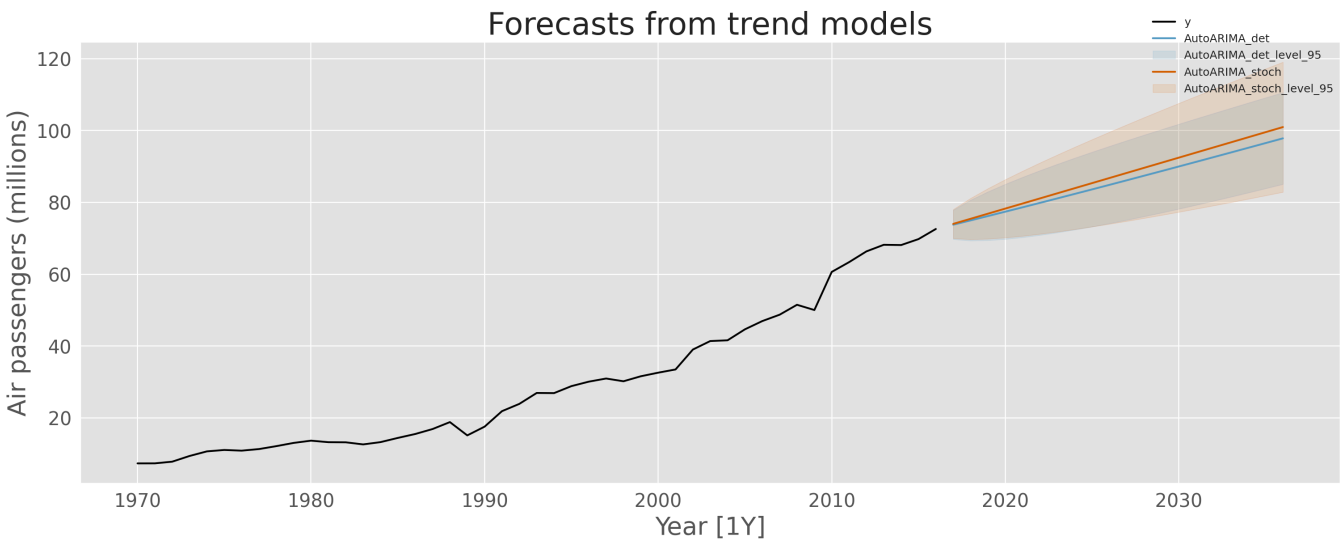

Figure 10.10: Forecasts of annual passengers for Australian air carriers using a deterministic trend model (blue) and a stochastic trend model (orange).

There is an implicit assumption with deterministic trends that the slope of the trend is not going to change over time. On the other hand, stochastic trends can change, and the estimated growth is only assumed to be the average growth over the historical period, not necessarily the rate of growth that will be observed into the future. Consequently, it is safer to forecast with stochastic trends, especially for longer forecast horizons.

10.5 Dynamic harmonic regression

When there are long seasonal periods, a dynamic regression with Fourier terms is often better than other models we have considered in this book.[2]

For example, daily data can have annual seasonality of length 365, weekly data has seasonal period of approximately 52, while half-hourly data can have several seasonal periods, the shortest of which is the daily pattern of period 48.

Seasonal versions of ARIMA and ETS models are designed for shorter periods such as 12 for monthly data or 4 for quarterly data. The `ETS()` model restricts seasonality to be a maximum period of 24 to allow hourly data but not data with a larger seasonal period. The problem is that there are $m-1$ parameters to be estimated for the initial seasonal states where m is the seasonal period. So for large m, the estimation becomes almost impossible.

So for such time series, we prefer a harmonic regression approach where the seasonal pattern is modelled using Fourier terms with short-term time series dynamics handled by an ARMA error.

The advantages of this approach are:

- it allows any length seasonality;
- for data with more than one seasonal period, Fourier terms of different frequencies can be included;
- the smoothness of the seasonal pattern can be controlled by K, the number of Fourier sin and cos pairs – the seasonal pattern is smoother for smaller values of K;
- the short-term dynamics are easily handled with a simple ARMA error.

The only real disadvantage (compared to a seasonal ARIMA model) is that the seasonality is assumed to be fixed — the seasonal pattern is not allowed to change over time. But in practice, seasonality is usually remarkably constant so this is not a big disadvantage except for long time series.

Example: Australian eating out expenditure

In this example we demonstrate combining Fourier terms for capturing seasonality with ARIMA errors capturing other dynamics in the data. For simplicity, we will use an example with monthly data. The same modelling approach using weekly data is discussed in Section 13.1.

We use the total monthly expenditure on cafes, restaurants and takeaway food services in Australia (\$ billion) from 2004 up to 2018 and forecast 24 months ahead. We vary K, the number of Fourier sin and cos pairs, from $K=1$ to $K=6$ (which is equivalent to including seasonal dummies). Figure 10.11 shows the seasonal pattern projected forward as K increases. Notice that as K increases the Fourier terms capture and project a more “wiggly” seasonal pattern and simpler ARIMA models are required to capture other dynamics. The AICc value is minimised for $K=6$, with a significant jump going from $K=4$ to $K=5$, hence the forecasts generated from this model would be the ones used.

```
industry = "Cafes, restaurants and takeaway food services"
aus_cafe_df = (
    pd.read_csv("../data/aus_retail.csv", parse_dates=["Month"])
    .loc[lambda x: x["Month"].between("2004", "2018-12-01")
        & (x["Industry"] == industry)
    ]
    .rename(columns={"Month": "ds", "Turnover": "y"})
    .groupby("ds", as_index=False)
    .agg({"y": "sum"})
    .assign(unique_id="aus_cafe")
)

def arima_with_fourier(k):
    aus_cafe = aus_cafe_df.assign(y=np.log(aus_cafe_df["y"]))
    features = [partial(fourier, season_length=12, k=k)]
    aus_cafe, fut = pipeline(
        aus_cafe, features=features, freq="MS", h=24)
    model = AutoARIMA(season_length=12, seasonal=False)
    sf = StatsForecast(models=[model], freq="MS")
    result = sf.forecast(df=aus_cafe, h=24, X_df=fut, level=[80, 95])
    numerics = result.select_dtypes(include=np.number)
    result[numerics.columns] = numerics.apply(np.exp)
    result["unique_id"] = "K=" + str(k)
    return result

def plot_with_intervals(ax, result, color):
    lo, mid, hi = "AutoARIMA-lo-95", "AutoARIMA", "AutoARIMA-hi-95"
    sns.lineplot(data=aus_cafe_df, x="ds", y="y", ax=ax, color="black")
    sns.lineplot(data=result, x="ds", y=mid, ax=ax, color=color)
    sns.lineplot(data=result, x="ds", y=lo, ax=ax, color=color, alpha=0.2)
    sns.lineplot(data=result, x="ds", y=hi, ax=ax, color=color, alpha=0.2)
    ax.fill_between(result["ds"], result[lo], result[hi],
        color=color, alpha=0.2)
    ax.set_ylabel("")
    ax.set_xlabel("")
    ax.set_title("K = " + str(i + 1))

fig, axes = plt.subplots(3, 2, figsize=(8, 8))
colors = ["#d6610a", "#207fb9", "#0aa076", "#d388ac", "#e9a32b", "#62b9e9"]
for i, (ax, color) in enumerate(zip(axes.flat, colors)):
    result = arima_with_fourier(i + 1)
    plot_with_intervals(ax, result, color)
fig.suptitle("Total monthly eating-out expenditure")
fig.supylabel("$ billions")
fig.supxlabel("Month")
plt.show()
```

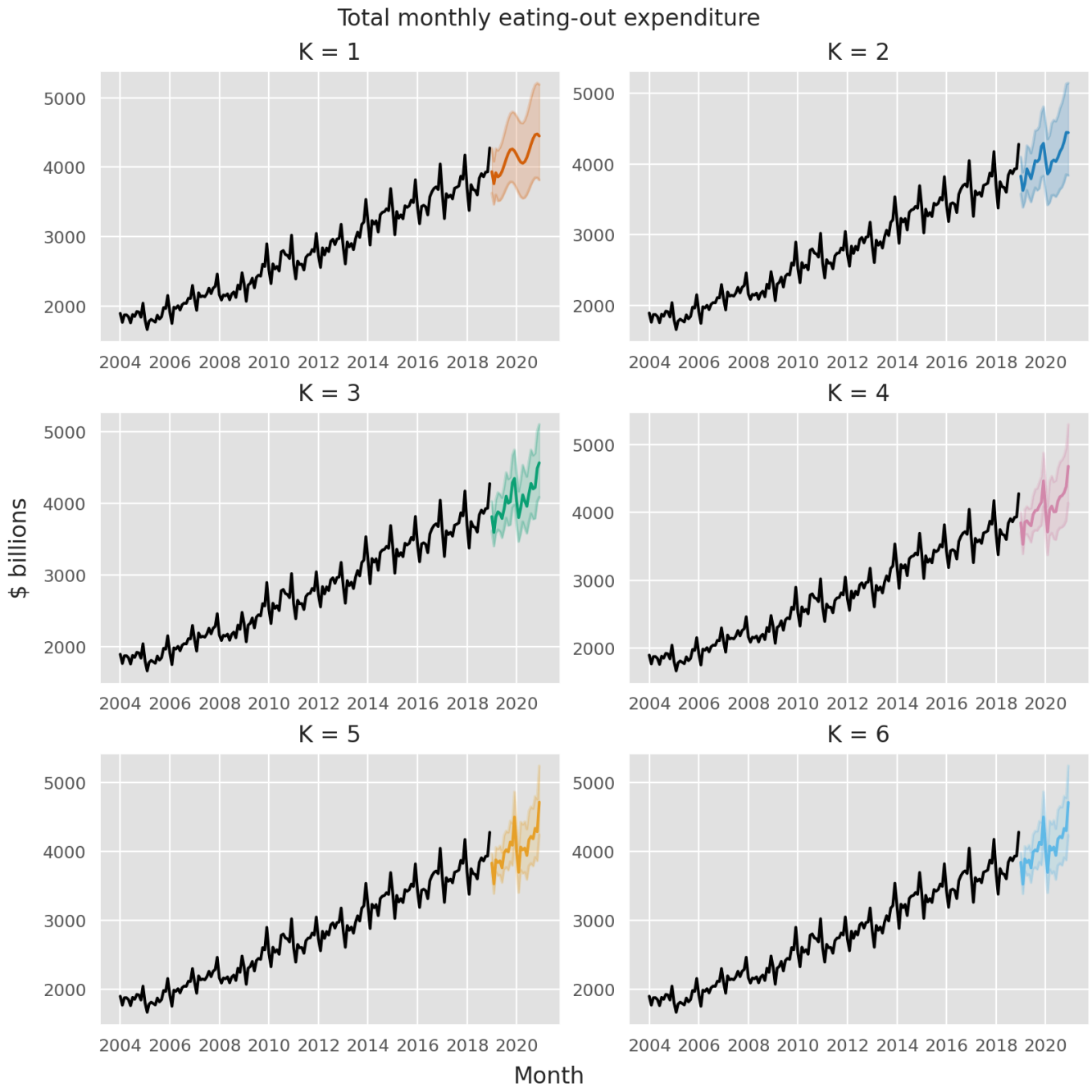

Figure 10.11: Using Fourier terms and ARIMA errors for forecasting monthly expenditure on eating out in Australia.

10.6 Lagged predictors

Sometimes, the impact of a predictor that is included in a regression model will not be simple and immediate. For example, an advertising campaign may impact sales for some time beyond the end of the campaign, and sales in one month will depend on the advertising expenditure in each of the past few months. Similarly, a change in a company's safety policy may reduce accidents immediately, but have a diminishing effect over time as employees take less care when they become familiar with the new working conditions.

In these situations, we need to allow for lagged effects of the predictor. Suppose that we have only one predictor in our model. Then a model which allows for lagged effects can be written as

$$y_t = \beta_0 + \gamma_0 x_t + \gamma_1 x_{t-1} + \cdots + \gamma_k x_{t-k} + \eta_t,$$

where η_t is an ARIMA process. The value of k can be selected using the AICc, along with the values of p and q for the ARIMA error.

Example: TV advertising and insurance quotations

A US insurance company advertises on national television in an attempt to increase the number of insurance quotations provided (and consequently the number of new policies). Figure 10.12 shows the number of quotations and the expenditure on television advertising for the company each month from January 2002 to April 2005.

```
insurance = pd.read_csv('../data/insurance.csv', parse_dates=['ds'])
fig, axes = plt.subplots(2, sharex=True)
sns.lineplot(data=insurance, x="ds", y="Quotes",
    ax=axes[0], color="black")
sns.lineplot(data=insurance, x='ds', y='TVadverts',
    ax=axes[1], color='black')
axes[1].set(xlabel="Month")
fig.suptitle("Insurance advertising and quotations")
plt.show()
```

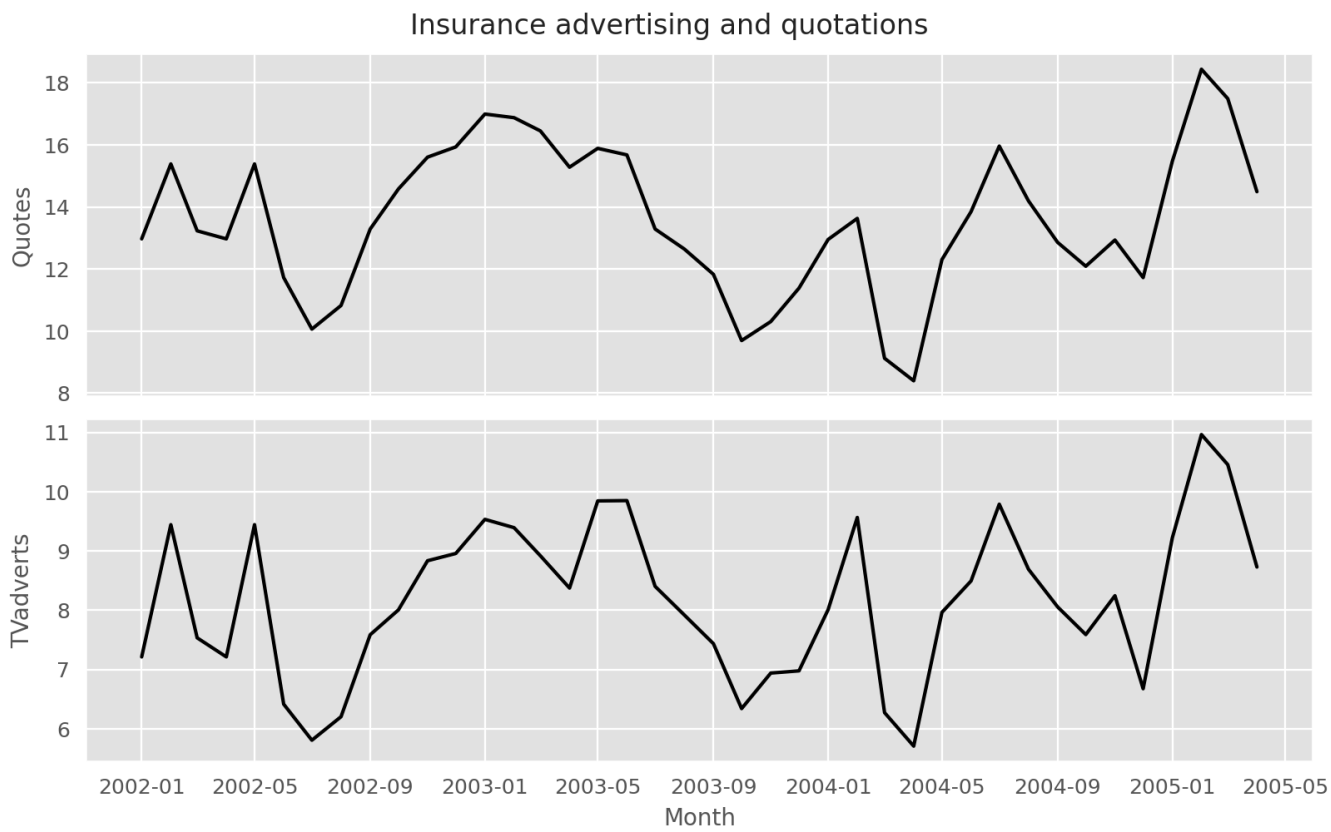

Figure 10.12: Numbers of insurance quotations provided per month and the expenditure on advertising per month.

We will consider including advertising expenditure for up to four months; that is, the model may include advertising expenditure in the current month, and the three months before that. When comparing models, it is important that they all use the same training set. In the following code, we exclude the first three months in order to make fair comparisons.

```
train = (
    insurance.assign(**{
        f"TVadverts_{lag}": insurance["TVadverts"].shift(lag)
        for lag in (1, 2, 3)
    })
    .assign(Quotes=insurance.Quotes.iloc[4:])
)

base_columns = ["unique_id", "ds", "Quotes", "TVadverts"]
lag_columns = ["TVadverts_1", "TVadverts_2", "TVadverts_3"]
model = AutoARIMA(season_length=12, d=0)
sfs = [
    StatsForecast(models=[model], freq='MS').fit(
        df=train[[*base_columns, *lag_columns[:n]]], target_col="Quotes")
    for n in range(4)
]
```

Next we choose the optimal lag length for advertising based on the AICc.

```
df = pd.DataFrame([
    {"model": f"lag{i}", "aicc": sf.fitted_[0, 0].model_["aicc"]}
    for (i, sf) in enumerate(sfs)
])
df.round(2)
```

	model	aicc
0	lag0	66.67
1	lag1	58.43
2	lag2	61.53
3	lag3	64.71

The best model (with the smallest AICc value) is `lag1` with two predictors; that is, it includes advertising only in the current month and the previous month. So we now re-estimate that model, but using all the available data.

```
fit_df = insurance.assign(**{
    f"TVadverts_{lag}": insurance["TVadverts"].shift(lag)
    for lag in (1, 2, 3)
})
sf = StatsForecast(models=[AutoARIMA(d=0)], freq='MS')
fit_columns = ["unique_id", "ds", "Quotes", "TVadverts", "TVadverts_1"]
sf.fit(df=fit_df[fit_columns].dropna(), target_col="Quotes")
```

```
model = sf.fitted_[0, 0].model_
items = list(model["coef"].items())
arma = pd.Series(dict(items[:2]), name="ARMA Coefficients").round(3)
regr = pd.Series(dict(items[2:]), name="Regression Coefficients").round(3)
metrics = pd.Series({
    "sigma^2": model["sigma2"], "loglik": model["loglik"],
    "aic": model["aic"], "aicc": model["aicc"], "bic": model["bic"],
}, name="Metrics").round(2)
print(ARIMASummary(model), "\n")
print(arma.to_frame(), "\n")
print(regr.to_frame(), "\n")
print(metrics.to_frame())
```

```
Regression with ARIMA(1,0,2) errors

     ARMA Coefficients
ar1              0.512
ma1              0.917

           Regression Coefficients
ma2                          0.459
intercept                    2.155
ex_1                         1.253
ex_2                         0.146

         Metrics
sigma^2     0.22
loglik    -23.94
aic        61.88
aicc       65.49
bic        73.52
```

The chosen model has ARIMA(1,0,2) errors. The model can be written as

$$y_t = 2.155 + 1.253x_t + 0.146x_{t-1} + \eta_t,$$

where y_t is the number of quotations provided in month t, x_t is the advertising expenditure in month t,

$$\eta_t = 0.512\eta_{t-1} + \varepsilon_t + 0.917\varepsilon_{t-1} + 0.459\varepsilon_{t-2}$$

and ε_t is white noise.

We can calculate forecasts using this model if we assume future values for the advertising variable. If we set the future monthly advertising to 8 units, we get the forecasts in Figure 10.13.

```
uids = insurance["unique_id"].unique()
last_times = insurance[["ds"]].max()
future_predictors = (
    make_future_dataframe(uids, last_times, freq="MS", h=12)
    .assign(TVadverts=8, TVadverts_1=8)
)
sf.fit(df=fit_df[fit_columns].dropna(), target_col="Quotes")
```

```
fc = sf.predict(h=12, X_df=future_predictors, level=[80,95])
plot_series(insurance, fc, level=[80,95], target_col="Quotes",
    xlabel="Month", ylabel="Quotes",
    title="Forecast quotes with future advertising set to 8",
    palette="black_and_blue", rm_legend=False)
```

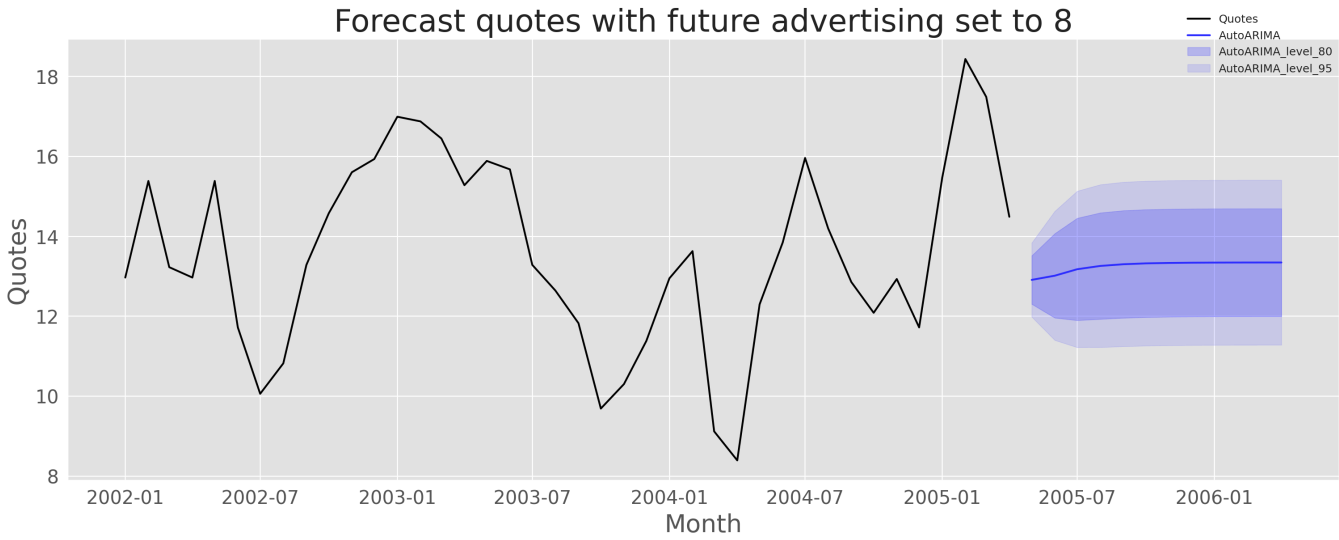

Figure 10.13: Forecasts of monthly insurance quotes, assuming that the future advertising expenditure is 8 units in each future month.

10.7 Exercises

1. This exercise uses data set `LakeHuron` giving the level of Lake Huron from 1875–1972.

   a. Load and plot the data.

   ```
   df = pd.read_csv('../data/lake_huron.csv', parse_dates=['ds'])
   ```

   b. Fit a linear trend model to the Lake Huron data with an ARMA error structure.

   c. Forecast the level for the next 30 years. Do you think the extrapolated linear trend is realistic?

2. Repeat Exercise 4 from Section 7.10, but this time adding in ARIMA errors to address the autocorrelations in the residuals.

   a. How much difference does the ARIMA error process make to the forecasts?
   b. Check the residuals of the ARIMA model to ensure it has adequately addressed the autocorrelations in the data.

3. Repeat the daily electricity example, but add an indicator variable called `Temperature2` with a value of 1 when the temperature is equal or greater than 25 degree Celsius, and zero otherwise.

```
vic_elec_daily['Temperature2'] = \
  (vic_elec_daily['Temperature'] >= 25).astype(int)
```

4. This exercise concerns `aus_accommodation`: the total quarterly takings from accommodation and the room occupancy level for hotels, motels, and guest houses in Australia, between January 1998 and June 2016. Total quarterly takings are in millions of Australian dollars.

   a. Compute the CPI-adjusted takings and plot the result for each state
   b. For each state, fit a dynamic regression model of CPI-adjusted takings with seasonal dummy variables, a linear trend model and ARIMA errors.
   c. Check that the residuals of the ARIMA model look like white noise.
   d. Forecast the takings for each state to the end of 2017. (Hint: You will need to produce forecasts of the CPI first.)
   e. What sources of uncertainty have not been taken into account in the prediction intervals?

5. We fitted a harmonic regression model to part of the `us_gasoline` series in Exercise 5 in Section 7.10. We will now revisit this model, and extend it to include more data and ARMA errors.

   a. Using `MLForecast`, fit a harmonic regression with a trend to the full series. Select the appropriate number of Fourier terms empirically.
   b. Now include ARIMA errors, keeping the same predictor variables as you used before.
   c. Check the residuals of the ARIMA model. Do they look sufficiently like white noise to continue? If not, try modifying your model, or removing the first few years of data.
   d. Once you have a model with white noise residuals, produce forecasts for the next year.

6. Electricity consumption is often modelled as a function of temperature. Temperature is measured by daily heating degrees and cooling degrees. Heating degrees is 18°C minus the average daily temperature when the daily average is below 18°C; otherwise it is zero. This provides a measure of our need to heat ourselves as temperature falls. Cooling degrees measures our need to cool ourselves as the temperature rises. It is defined as the average daily temperature minus 18°C when the daily average is above 18°C; otherwise it is zero. Let y_t denote the monthly total of kilowatt-hours of electricity used, let $x_{1,t}$seja o total mensal de graus de aquecimento, e seja$x_{2\,,\,t}$denota o total mensal de graus de resfriamento.

   Um analista ajusta o seguinte modelo a um conjunto desses dados:

$$y_t^* = \beta_1 x_{1\,,\,t}^* + \beta_2 x_{2\,,\,t}^* + \eta_t,$$

   onde

$$\left(1 - \Phi_1 B^{12} - \Phi_2 B^{24}\right)\left(1 - B\right)\left(1 - B^{12}\right)\eta_t = \left(1 + \theta_1 B\right)\varepsilon_t$$

e$y_t^* = \log(y_t)$,$x_{1,t}^* = \sqrt{x_{1,t}}ex_{2,t}^* = \sqrt{x_{2,t}}$.

a. Que tipo de modelo ARIMA foi identificado paraη_t?

b. Os coeficientes estimados são

Parâmetro	Estimativa	se	Z	P-valor
β_1	0,0077	0,0015	4,98	0,000
β_2	0,0208	0,0023	9.23	0,000
θ_1	-0,5830	0,0720	8.10	0,000
Φ_1	-0,5373	0,0856	-6,27	0,000
Φ_2	-0,4667	0,0862	-5,41	0,000

Explique o que são as estimativas deβ_1eβ_2Fale-nos sobre o consumo de eletricidade.

c. Escreva a equação em uma forma mais adequada para previsão.

d. Descreva como esse modelo poderia ser usado para prever a demanda de eletricidade para os próximos 12 meses.

e. Explique por que oη_tO termo deve ser modelado com um modelo ARIMA em vez de modelar os dados usando um pacote de regressão padrão. Em sua discussão, comente sobre as propriedades das estimativas, a validade dos resultados da regressão padrão e a importância do$\eta_t$modelo na produção de previsões.

7. Para as séries temporais de varejo consideradas em capítulos anteriores:

a. Desenvolva um modelo de regressão dinâmica apropriado com termos de Fourier para a sazonalidade. (Você provavelmente precisará usar a mesma transformação de Box-Cox que identificou anteriormente.)

b. Verifique os resíduos do modelo ARIMA. A série de resíduos se assemelha a ruído branco?

c. Compare as previsões com aquelas que você obteve anteriormente usando modelos alternativos.

10.8 Leitura complementar

- Uma discussão detalhada de modelos de regressão dinâmica é fornecida em Pankratz (1991) .
- Uma generalização de modelos de regressão dinâmica, conhecidos como “modelos de função de transferência”, é discutida em Box et al. (2015) .

10.9 Módulos e classes utilizados

Previsão de MLF

- `MLForecast` Classe - Para previsão baseada em aprendizado de máquina

Previsão de Estatísticas

- `StatsForecast` Classe - Motor de previsão principal
- `AutoARIMA()` Modelo - Para modelagem ARIMA automática

Previsão de utilidades

- `trend()`, `fourier()` funções - Para engenharia de recursos
- `plot_series()` Utilitário - Para visualização de séries temporais

---

1. A previsão com modelos cointegrados é discutida por Harris e Sollis (2003) .

2. O termo “regressão harmônica dinâmica” também é usado para uma regressão harmônica com parâmetros variáveis no tempo (Young, Pedregal e Tych 1999) .

Publicado pela OTexts usando Quarto . © 2026